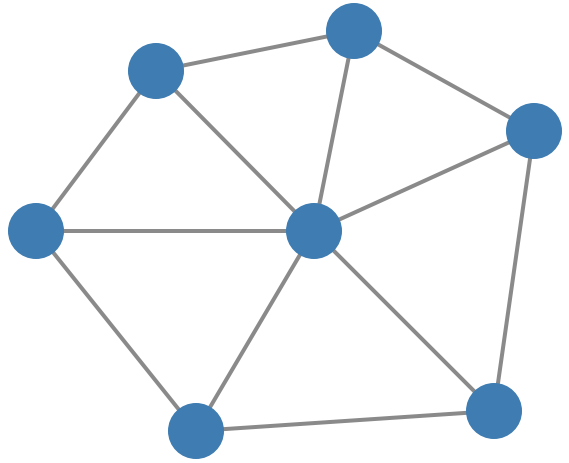
Graph Neural Networks

Foundations and applications for real-world networks



Presented by: Godbless James, Christian Nwandu, Nataram Odumo

August 2026 · Lagos, Nigeria

SECTION 1

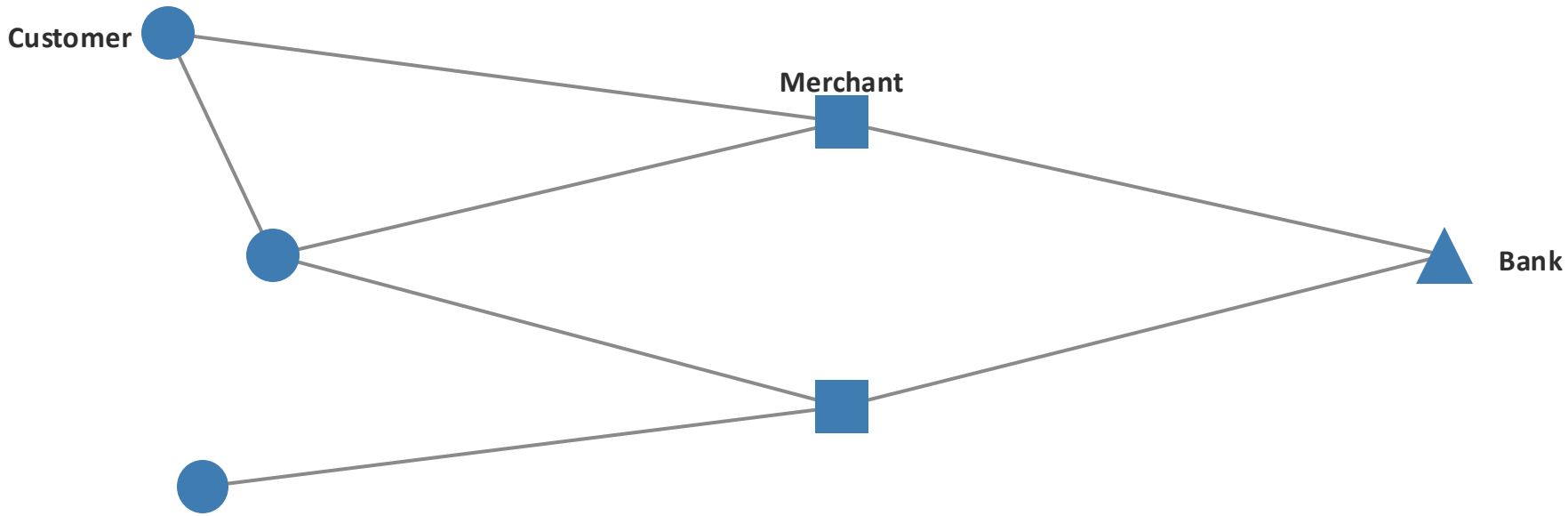
Why graphs matter

What a graph is, and why the world is full of them.

The world is more connected than a table can show.

Example: mobile money

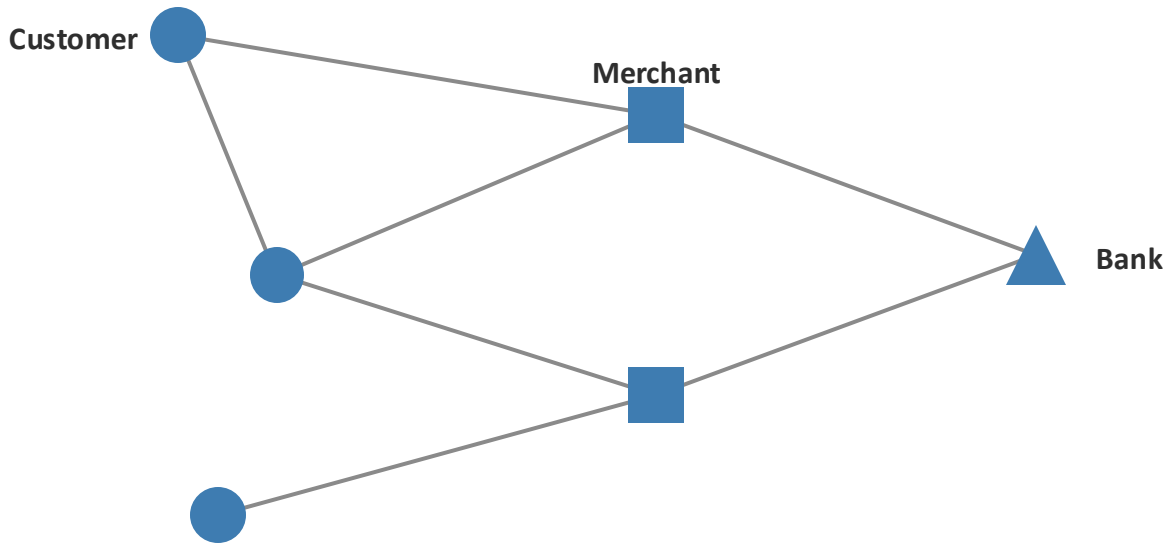
M-Pesa, MTN MoMo, Wave. Accounts, merchants, and banks, connected by transactions.



Real African systems are already networks. We just call them graphs when we study them.

Four things to notice

Same picture as before. Four ideas to hold onto.



1 **Things**

Accounts, merchants, and banks are the things.

2 **Connections**

A transaction links two things.

3 **Properties**

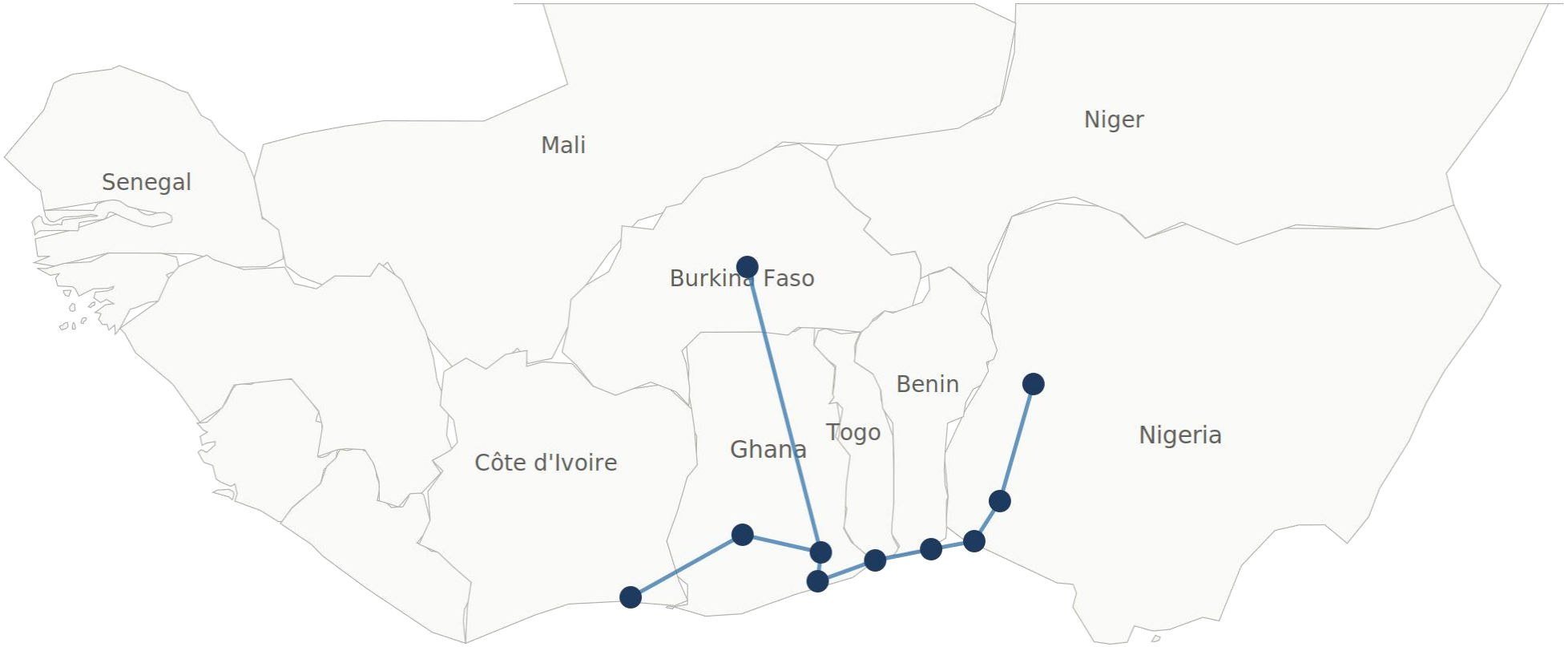
Things carry properties, like a balance or a location.

4 **Directions**

Transactions have a sender and a receiver.

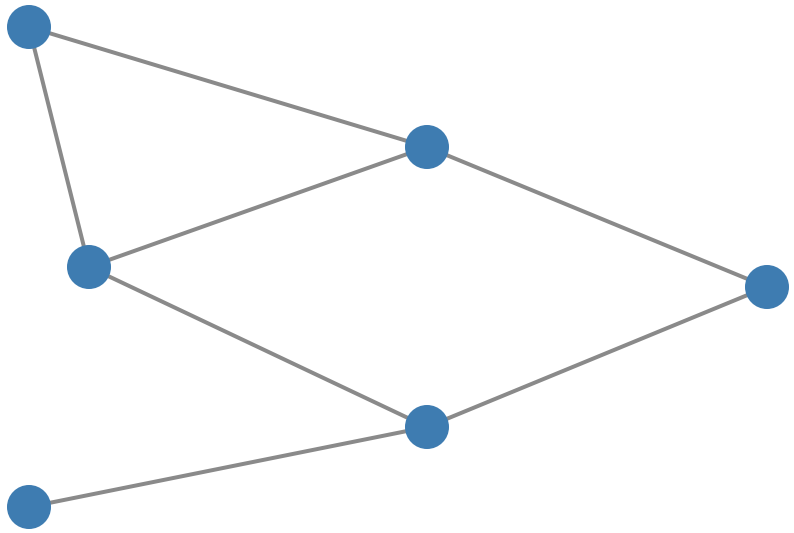
Example: the West African Power Pool

Substations across West Africa, connected by high-voltage transmission lines.

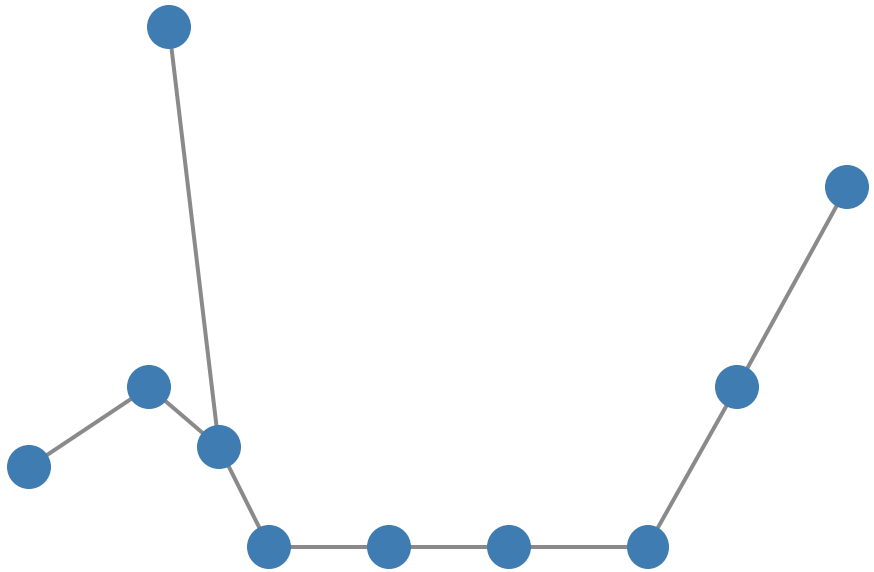


A different domain. The same structural idea.

Two systems, one shape



Mobile money

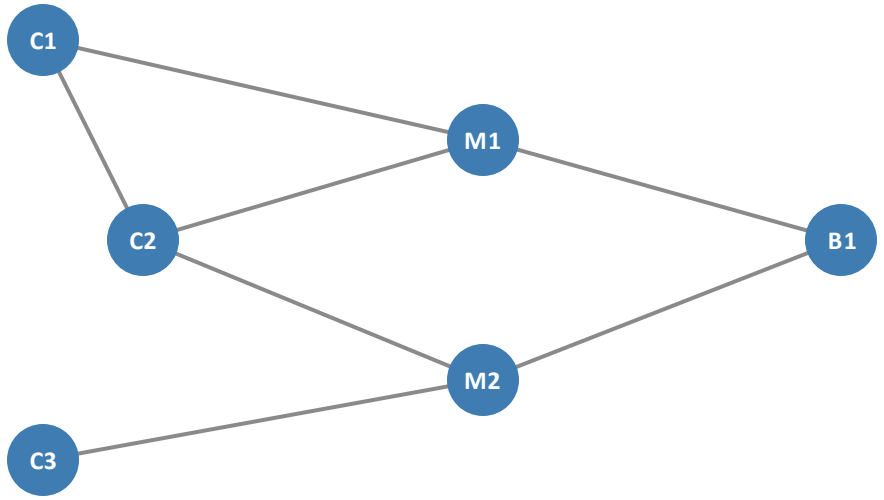


West African Power Pool

The same four ideas describe both: things, connections, properties, directions.

The connections disappear

Same data. Two representations.



Sender	Receiver	Amount	Time
C1	M1	200	09:12
C1	C2	500	09:34
C2	M1	150	10:07
C2	M2	80	10:32
C3	M2	220	11:45
M1	B1	350	12:00
M2	B1	180	12:15

The table has all the transactions. The graph has all the relationships.

Now let's give these ideas proper names.

SECTION 2

Graph fundamentals

The formal names for what we've been looking at.

The formal names

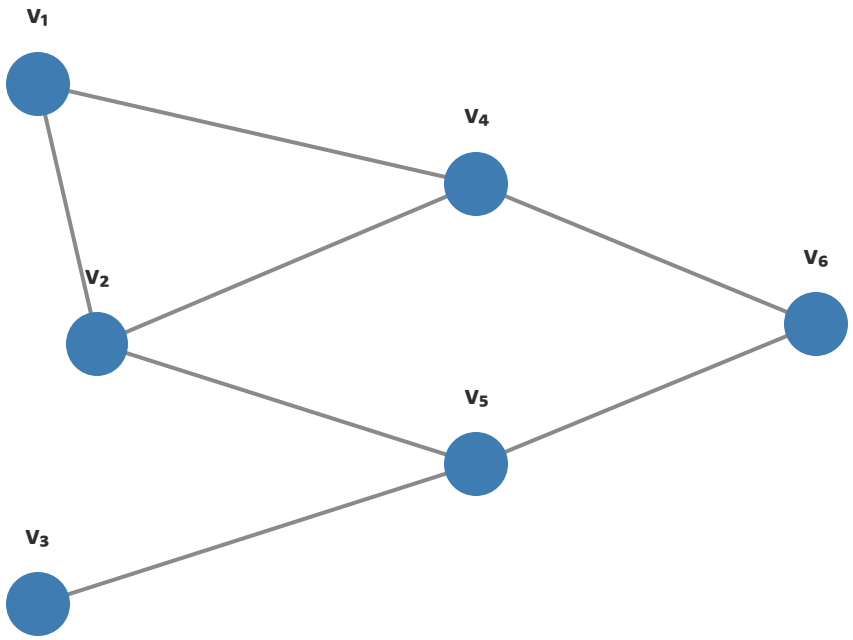
What we've been calling something has a proper term.

	INFORMAL	FORMAL
1	Things	**Nodes**
2	Connections	**Edges**
3	Properties	**Features**
4	Directions	**Directed edges**

With names, we can talk precisely. With precision, we can build.

Nodes

A node represents one thing in the system.



A node is one of the things in the graph.

$$V = \{v_1, v_2, \ldots, v_n\}$$

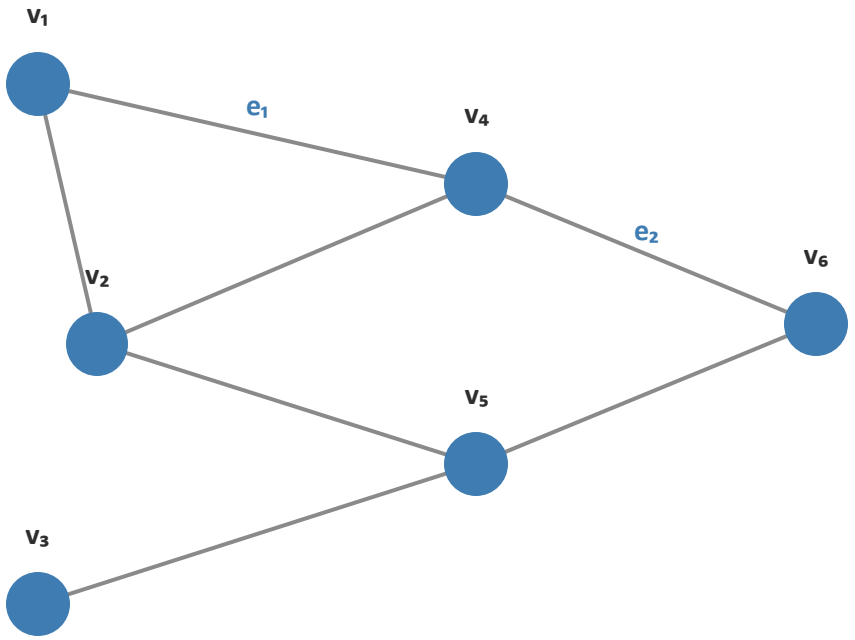
In our graph: |V| = 6.

Three customers, two merchants, one bank.

Every graph starts with its nodes.

Edges

An edge connects two nodes.



An edge is a connection between two nodes.

$$E = \{e_1, e_2, \ldots, e_m\}$$

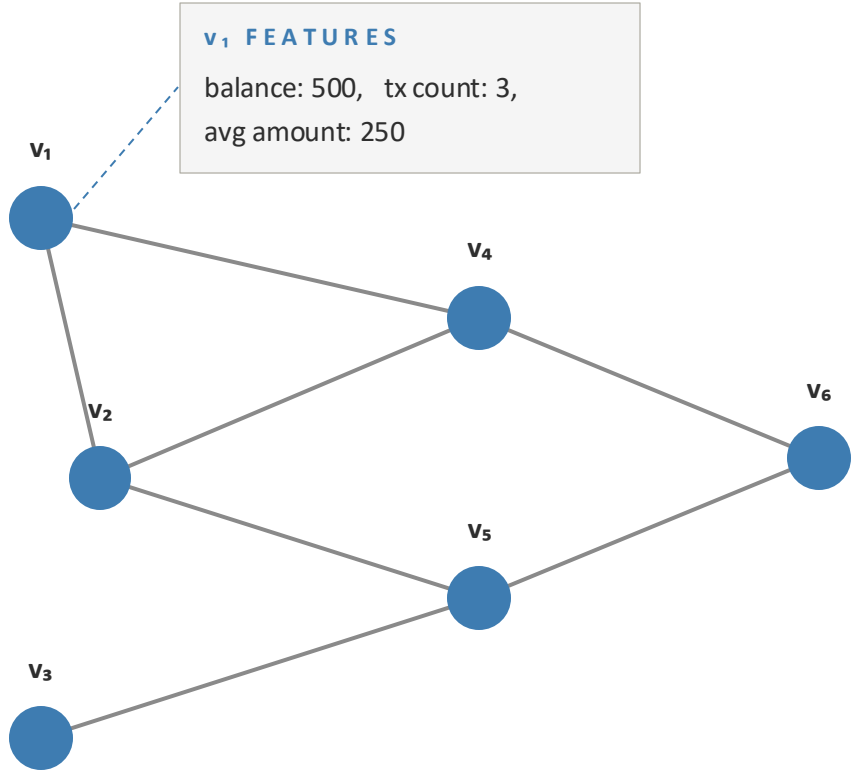
Each edge is a pair of nodes: e = (u, v).

In our graph: $e_1 = (v_1, v_4)$ and $|E| = 7$.

The edges are where the structure lives.

Node features

Each node carries its own data.



Each node has a feature vector.

$$x_v \in \mathbb{R}^d$$

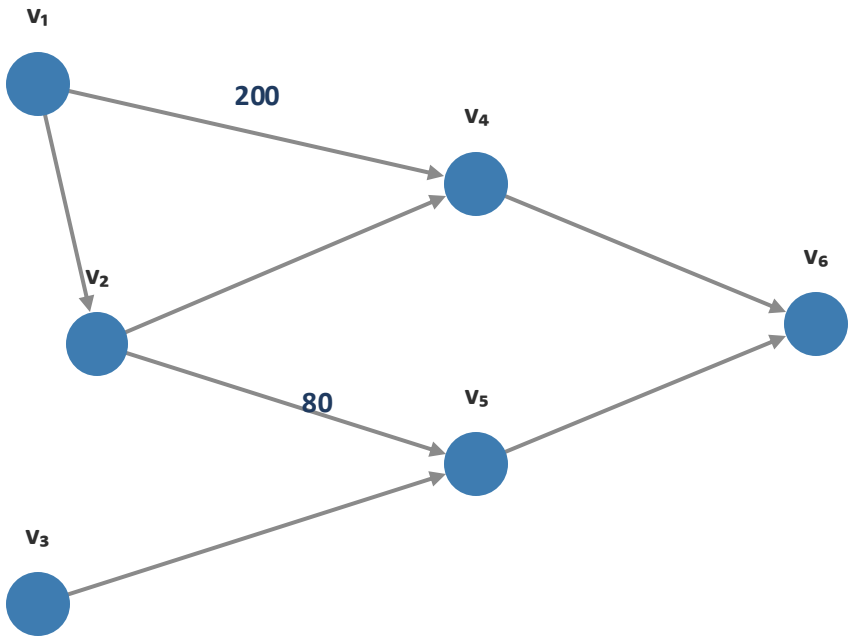
For v_1: $x_v = [500, 3, 250]$.

Three features: balance, tx count, avg amount.

A node's features are what the model sees.

Weighted and directed edges

Edges can carry a direction and a weight.



An edge can have a direction and a weight.

e = (u, v, w)

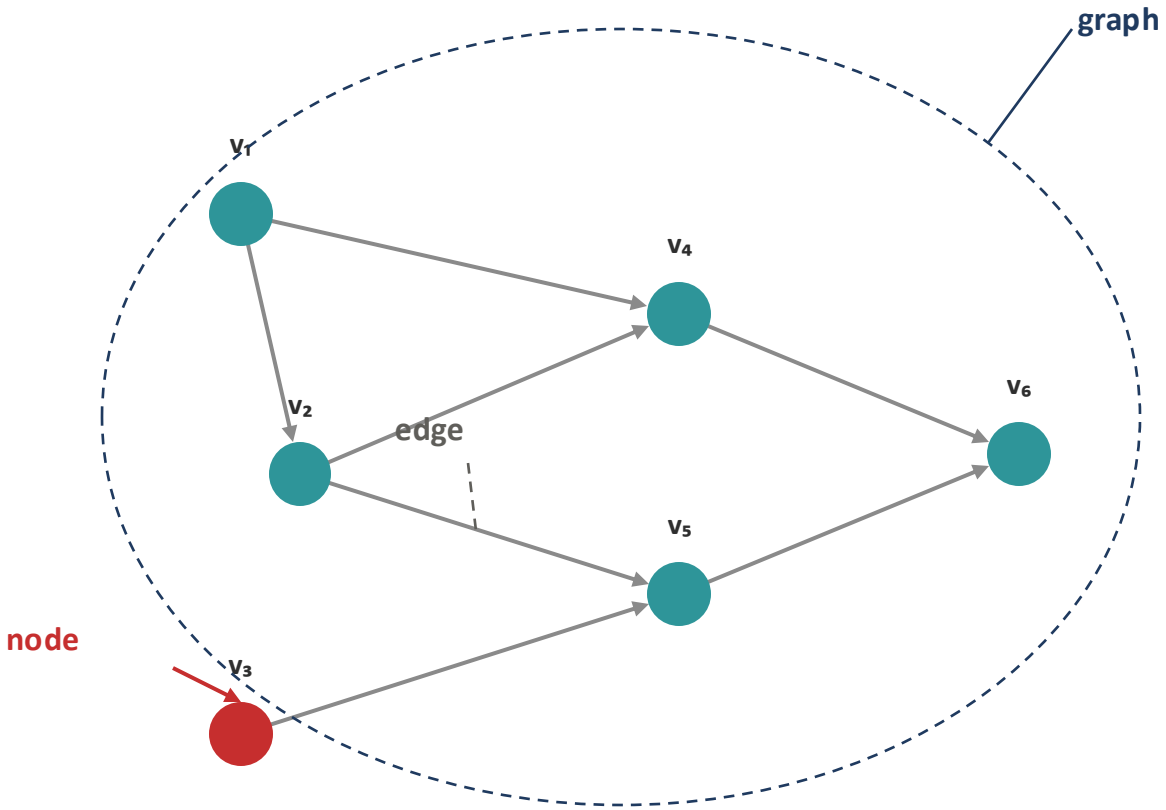
For our graph: (v_1, v_4, 200).

v_1 sent 200 to v_4.

An edge can say more than 'connected'.

Node, edge or graph?

The target can be a node, an edge, or a whole graph.



Node prediction

Predict a label for one node.

Is this account fraudulent?

Edge prediction

Predict whether a connection exists.

Will this account transact with this merchant?

Graph prediction

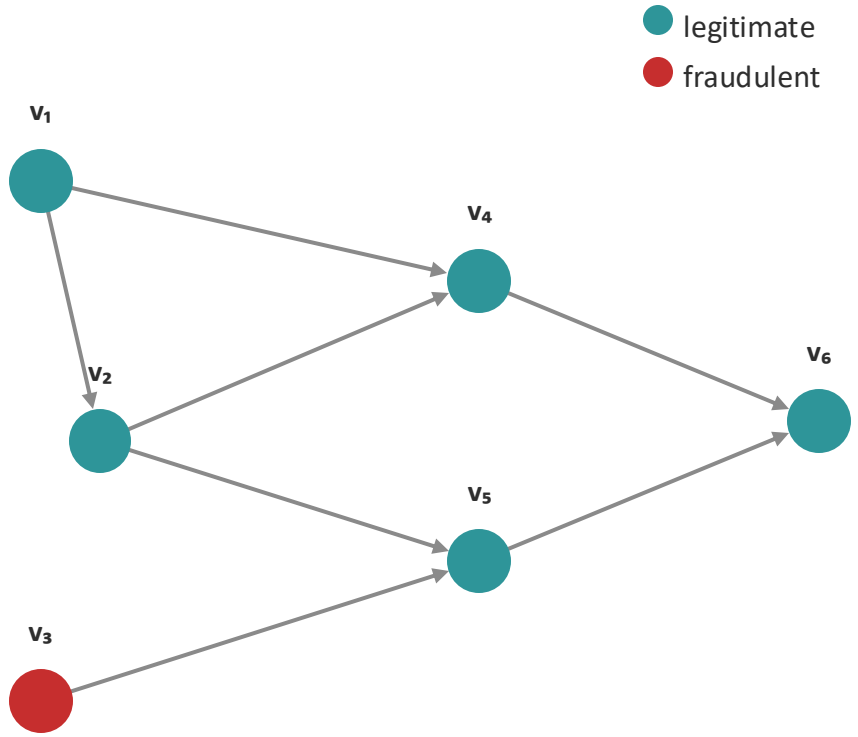
Predict one label for the whole graph.

Is this network snapshot unusually risky?

Today: node classification. One label per account.

The node classification label

What we want the model to predict for each node.



Each node has a label.

$$y_v \in \{0, 1\}$$

For v_3: $y_v = 1$ (fraudulent).

Every other node: $y_v = 0$ (legitimate).

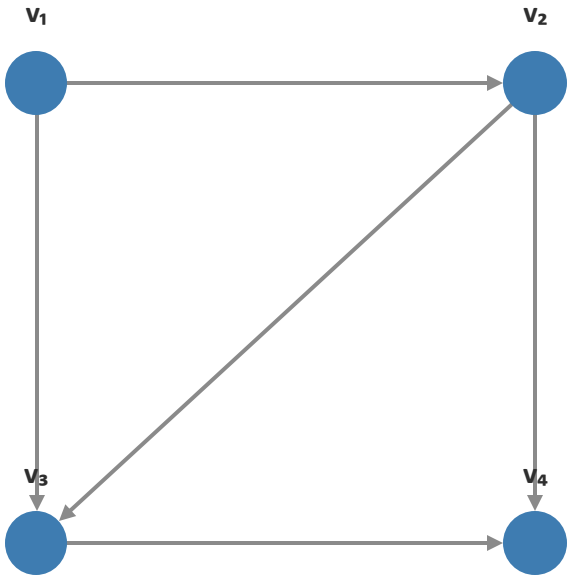
Labels are the target. The model learns to predict them.

We have the vocabulary.

Now we need the data structures.

Adjacency matrix

Every possible edge, present or not.



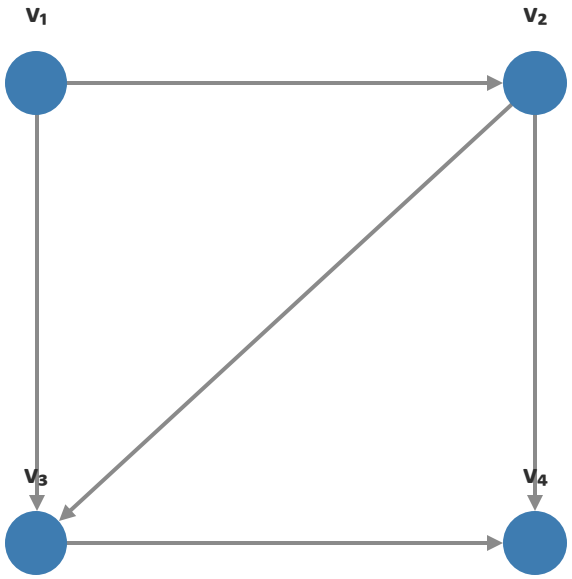
	v_1	v_2	v_3	v_4
v_1	0	**1**	1	0
v_2	0	0	1	1
v_3	0	0	0	1
v_4	0	0	0	0

A[i, j] = 1 means there's an edge from v_i to v_j. Highlighted: A[1, 2] = 1.

Easy to read. Big for large graphs.

Edge list

Just the edges that exist.



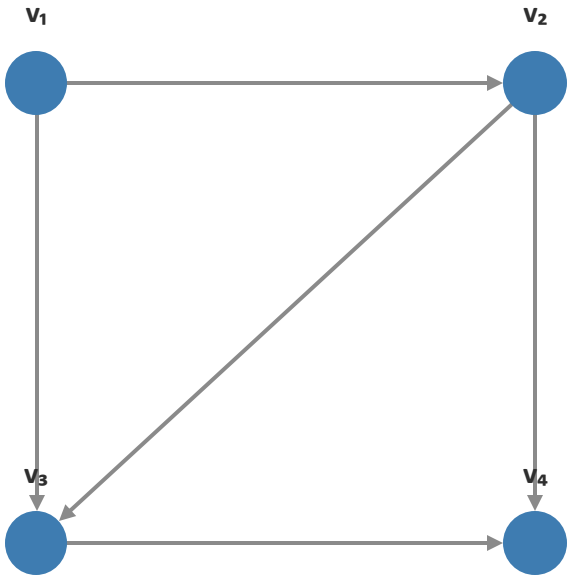
Source	Target
v_1	v_2
v_1	v_3
v_2	v_3
v_2	v_4
v_3	v_4

$|E|$ = 5 edges, 5 rows.

Only real edges take up space.

edge_index

The PyTorch Geometric convention.



source	**0**	0	1	1	2
target	**1**	2	2	3	3

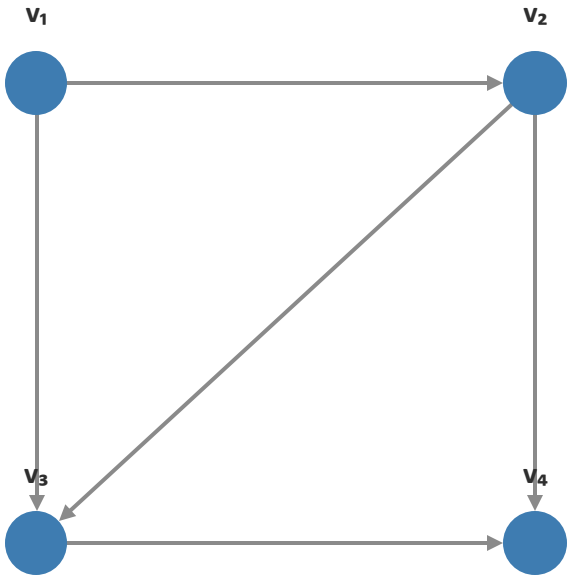
```
edge_index = torch.tensor([
    [0, 0, 1, 1, 2],   # sources
    [1, 2, 2, 3, 3],   # targets
])
```

In code: $v_1 = 0$, $v_2 = 1$, $v_3 = 2$, $v_4 = 3$.

The edge list, ready for a tensor library.

Node feature matrix

All node features stacked into one tensor.



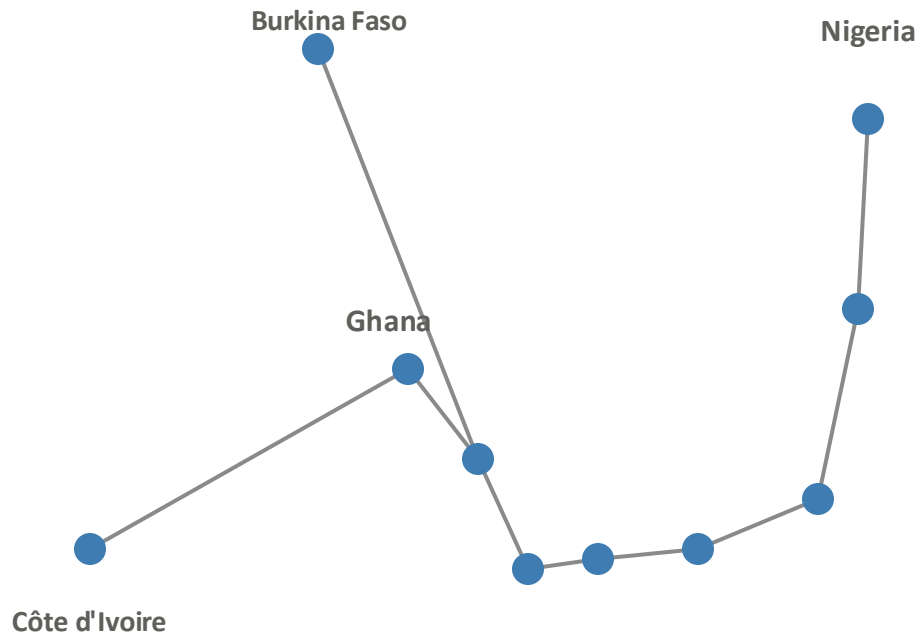
	balance	tx_count	avg_amount
v_1	**500**	**3**	**250**
v_2	800	5	400
v_3	150	1	150
v_4	3000	12	600

Row i is node i's feature vector. Highlighted: v_1 = [500, 3, 250].
Shape: (N, d) = (4, 3).

All node features in one tensor.

The same tools work everywhere

WAPP, as a graph object.



10 substations, 9 transmission lines.

Different domain. Same object.

edge_index

shape: (2, E) = (2, 9). One column per transmission line.

For Egbin → Ibadan: edge_index[:, 0] = [0, 1].

X

shape: (N, d) = (10, 3).

10 substations. 3 features each: capacity, voltage, type.

For Egbin (i = 0): X[0] = [500, 330, 0].

We can describe the graph.
Now let's learn from it.

SECTION 3

Graph Neural Networks

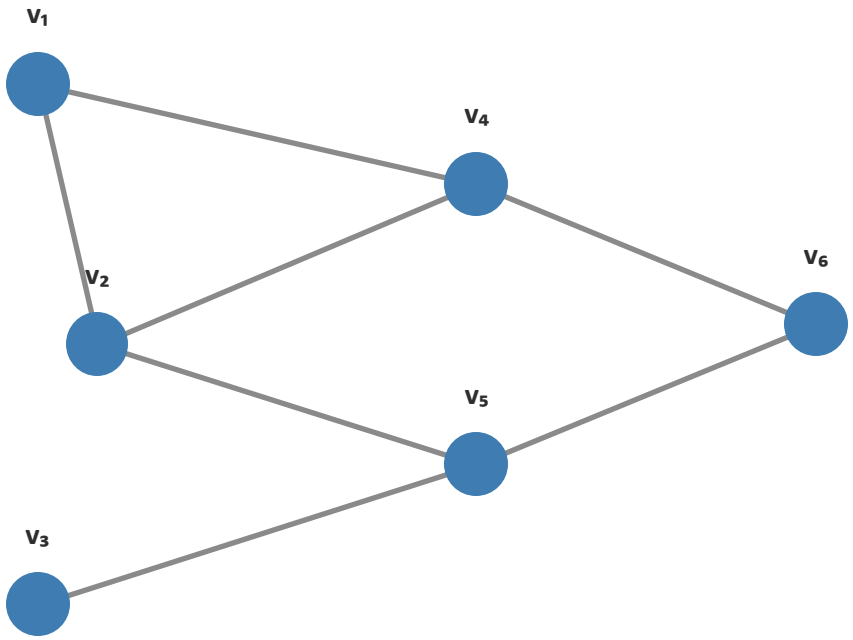
How a model learns from a graph.

So far, the edges have been decoration.

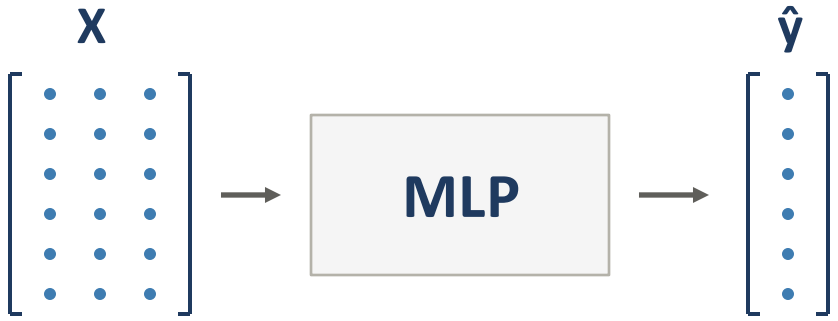
Now let's use them.

What a plain MLP sees

Node features. That's it.



edges faded — never read by the model

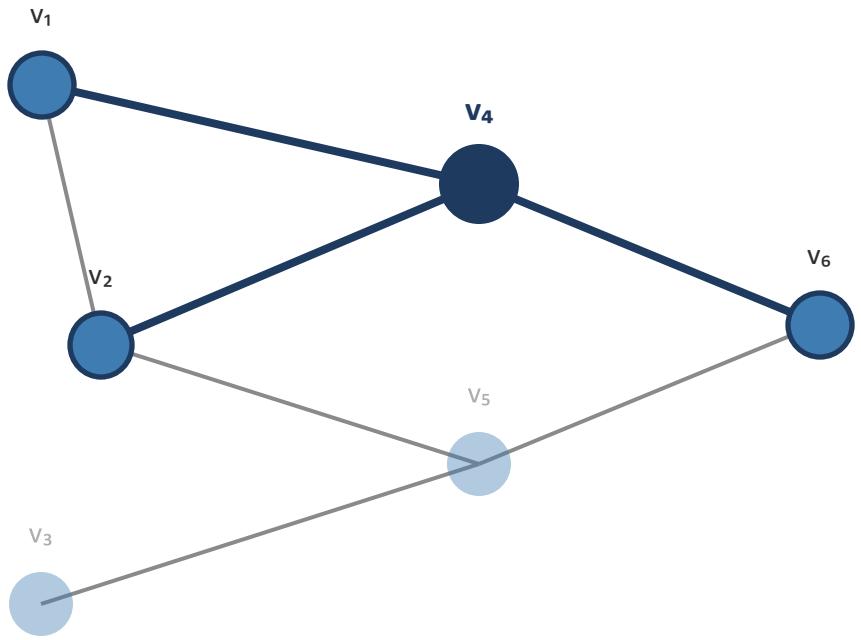


One row in, one prediction out. Rows never talk to each other.

Each account, on its own. No neighbourhood, no context.

Neighbours are evidence

The 1-hop neighbourhood is data the MLP threw away.



A node's neighbourhood is the set of nodes it's connected to.

$$N(v) = \{ u : (u, v) \in E \}$$

For v_4: $N(v_4) = \{v_1, v_2, v_6\}$.

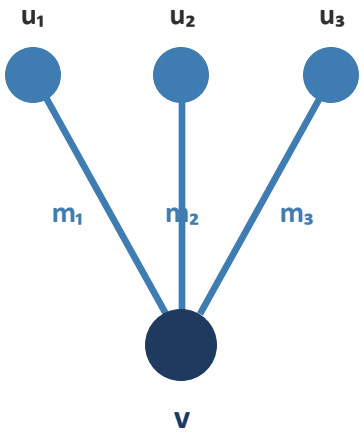
Three neighbours. A signal the MLP never reads.

The neighbourhood is a feature the MLP never sees.

Message passing

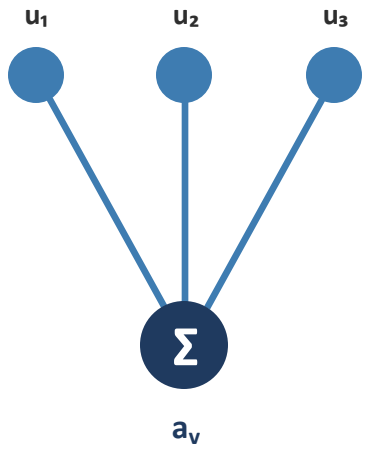
Every GNN layer does three things.

1 · MESSAGE



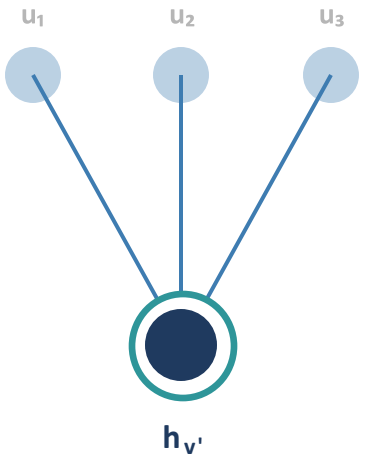
Each neighbour sends a message to v.

2 · AGGREGATE



Messages combine at v.

3 · UPDATE

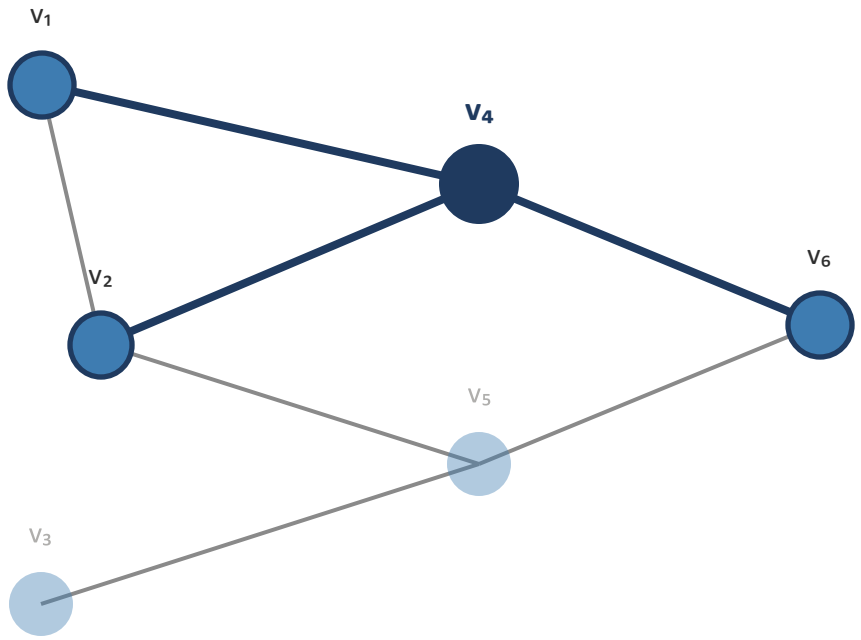


v updates its representation.

Every GNN layer is this frame.

Message passing on v_4

One step, in numbers.



STEP 1 · MESSAGES

$v_1 \rightarrow [500, 3, 200]$

$v_2 \rightarrow [800, 4, 300]$

$v_6 \rightarrow [1700, 5, 500]$

STEP 2 · AGGREGATE (SUM)

$\mathbf{a_{v_4} = [3000, 12, 1000]}$

STEP 3 · UPDATE

$x_{v_4} = [1000, 5, 400]$

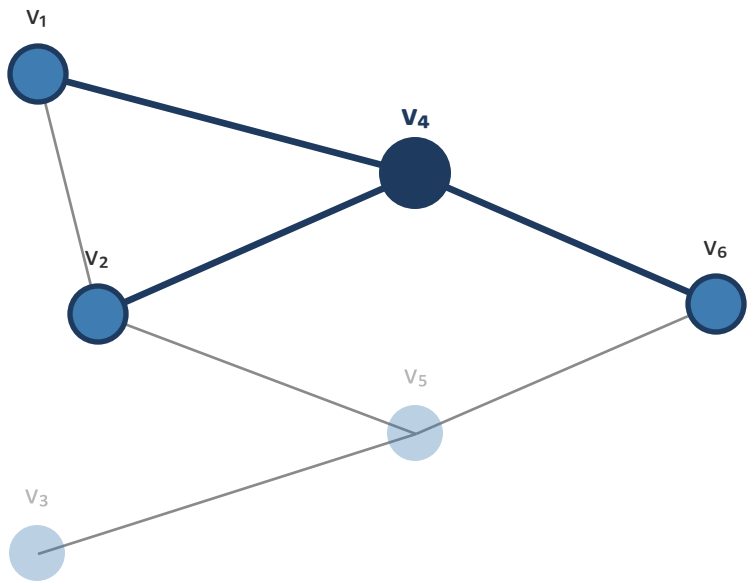
$\mathbf{h_{v_4'} = x_{v_4} + a_{v_4} = [4000, 17, 1400]}$

One layer. One hop deeper.

Stacking layers

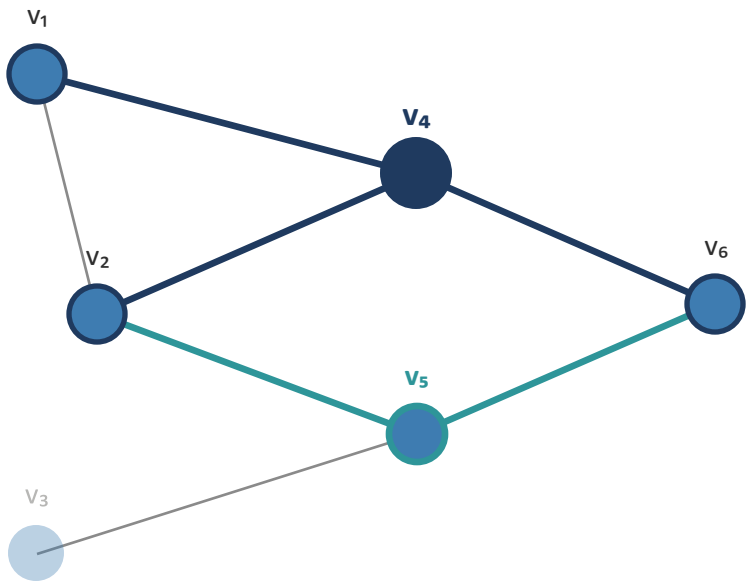
Each layer adds one hop of reach.

1 LAYER



v_4 hears from 3 neighbours.

2 LAYERS

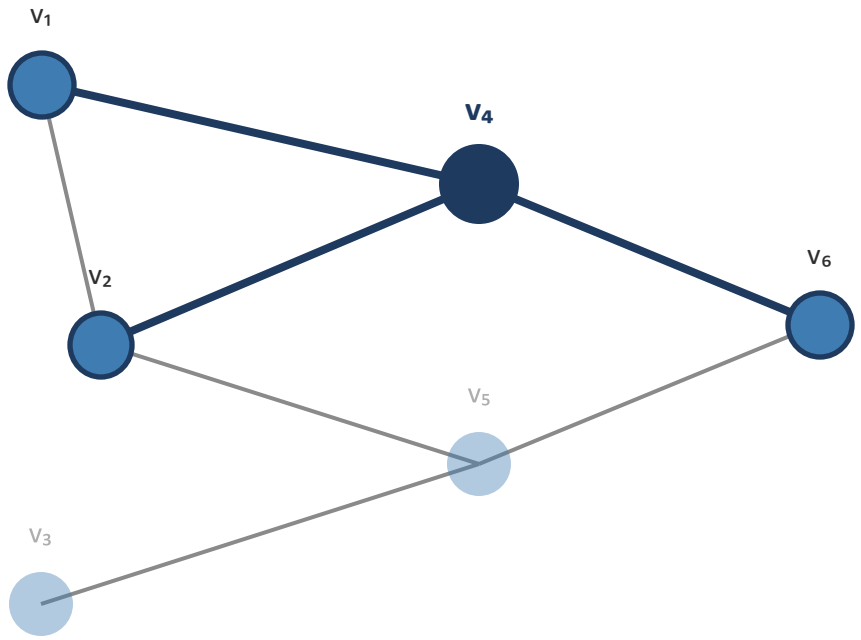


v_4 hears from 4 nodes. v_5 arrives through v_2 and v_6.

One layer, one hop. Stack more to reach further.

The GCN layer

Message passing with a specific choice of aggregate.



GCN UPDATE RULE

$$h'_v = \sigma\left(\sum W \cdot h_u / \sqrt{d_v \cdot d_u}\right)$$

over $u \in N(v) \cup \{v\}$ (neighbours + self)

W	learned weight matrix
$1/\sqrt{d_v \cdot d_u}$	normalise by node degrees
Σ	sum over neighbours and self
σ	non-linearity (typically ReLU)

GCN in one line: normalise, transform, activate.

The matrix form

GCN, expressed for all N nodes at once.

$$H^{(l+1)} = \sigma(\tilde{D}^{-1/2} \cdot \tilde{A} \cdot \tilde{D}^{-1/2} \cdot H^{(l)} \cdot W^{(l)})$$

Same math as Slide 32. Just written for the whole graph in one line.

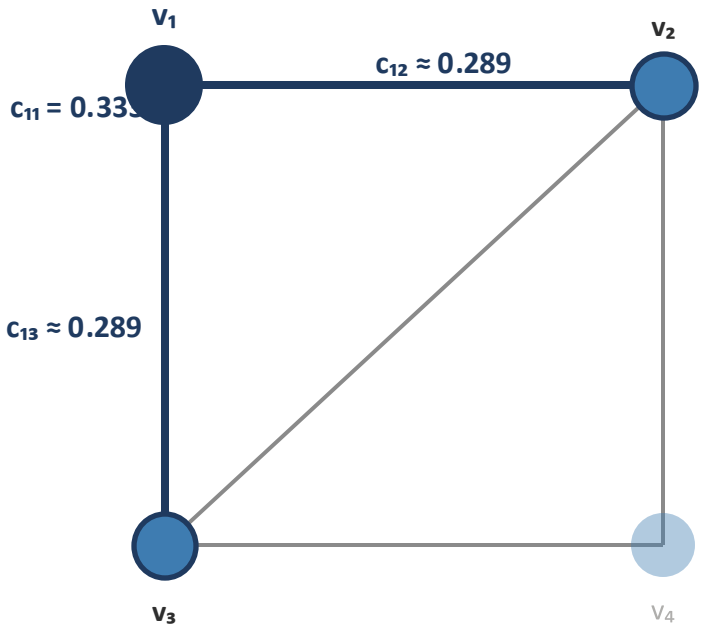
WHAT THE MATRICES MEAN

$\tilde{A} = A + I$	adjacency with self-loops — each node counts itself.
$\tilde{D}$	diagonal degree matrix of $\tilde{A}$.
$\tilde{D}^{-1/2} \cdot \tilde{A} \cdot \tilde{D}^{-1/2}$	symmetric normalisation — prevents high-degree domination.
$H^{(l)}$	all node features at layer l — shape (N, d).

This is what the library computes.

GCN on the toy graph

One node's update, coefficients from degrees.



v_1's UPDATE

$$h'_1 = \sigma\left(c_{11}\cdot W\cdot h_1 + c_{12}\cdot W\cdot h_2 + c_{13}\cdot W\cdot h_3 \right)$$

A weighted mix of transformed features from v_1 and its neighbours.

COEFFICIENTS FROM DEGREES

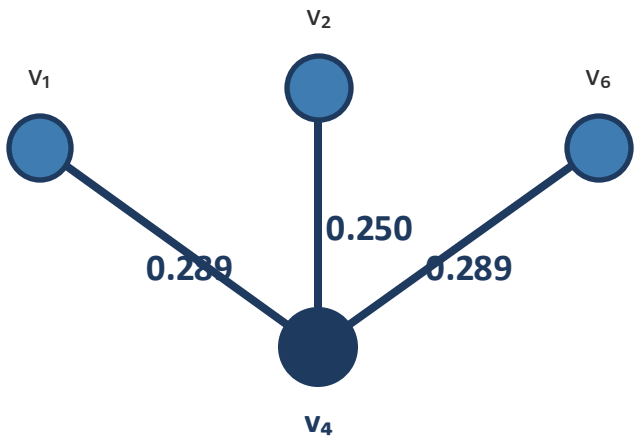
$c_{11} = 1/\sqrt{(3 \cdot 3)} = 0.333$	self-loop, $\tilde{d}_1 = 3$
$c_{12} = 1/\sqrt{(3 \cdot 4)} \approx 0.289$	from v_2, $\tilde{d}_2 = 4$
$c_{13} = 1/\sqrt{(3 \cdot 4)} \approx 0.289$	from v_3, $\tilde{d}_3 = 4$

One matrix multiply updates every node like this.

Not all neighbours are equal

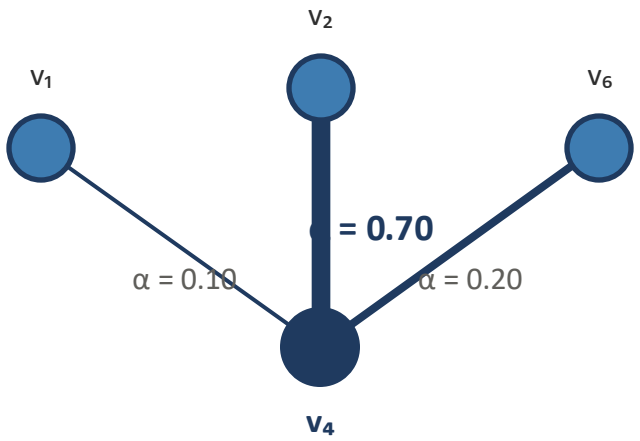
GCN's coefficients depend only on structure.

GCN



Coefficients fixed by degrees.

GAT

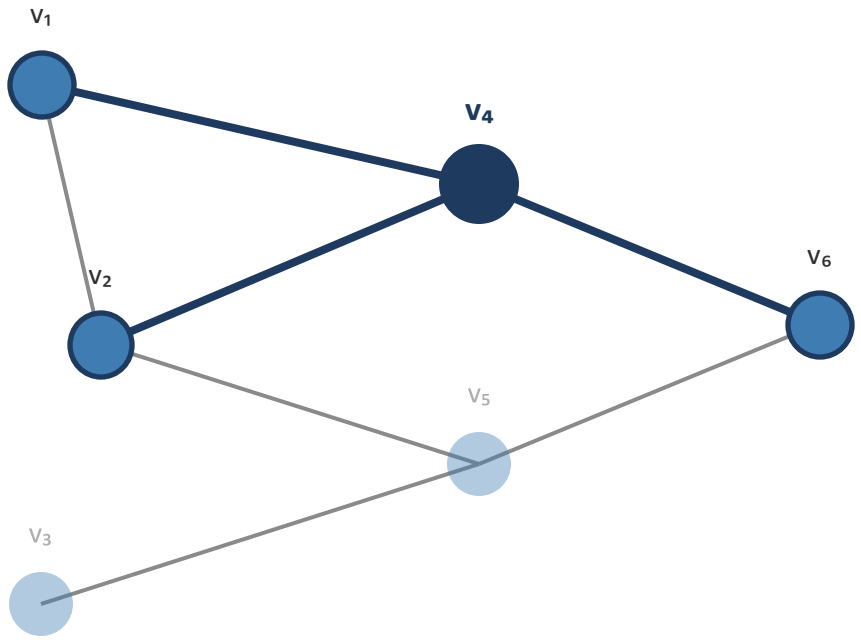


Attention learned per neighbour.

GAT lets the model decide who to listen to.

Attention

How GAT computes the weights.



Score from features. Normalise with softmax.

1 · SCORE

$$e_{ij} = \text{LeakyReLU}(a^T[\, W{\cdot}h_i \,\|\, W{\cdot}h_j \,])$$

For each edge (i, j).

2 · NORMALISE

$$\alpha_{ij} = \text{softmax}_i(e_{ij})$$

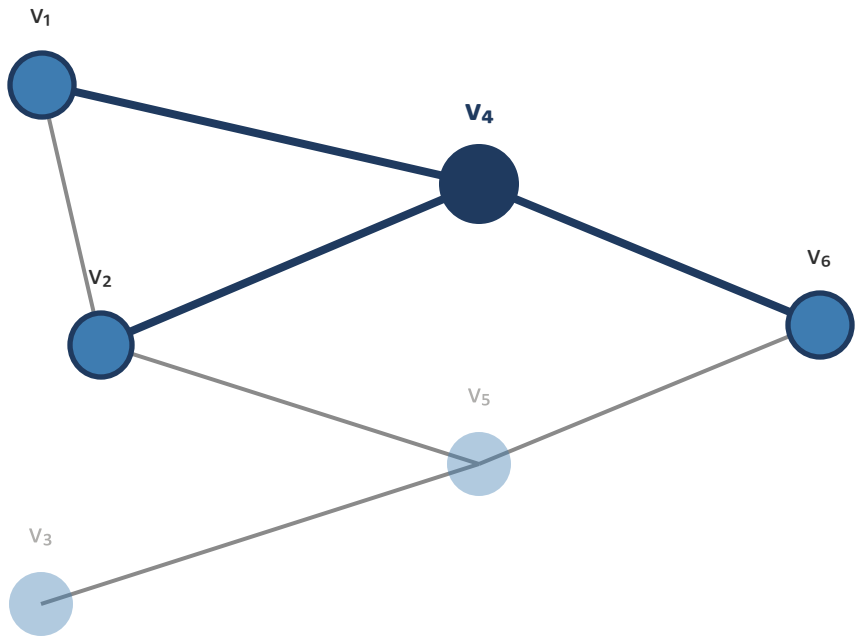
Over j ∈ N(i). Weights sum to 1.

COMPONENTS

a learned attention vector
W shared weight matrix
‖ vector concatenation

The GAT layer

Same shape as GCN. Different coefficient.



GCN (RECAP)

$$h'_v = \sigma\left(\sum (W \cdot h_u) / \sqrt{d_v \cdot d_u}\right)$$

GAT

$$h'_i = \sigma\left(\sum \alpha_{ij} \cdot W \cdot h_i\right)$$

Sum over j ∈ N(i) ∪ {i}. α_{ij} from the attention mechanism.

WHAT CHANGED

$1 / \sqrt{d_v \cdot d_u}$ → α_{ij}

fixed by degrees → learned from features

Same aggregation. Learned coefficients.

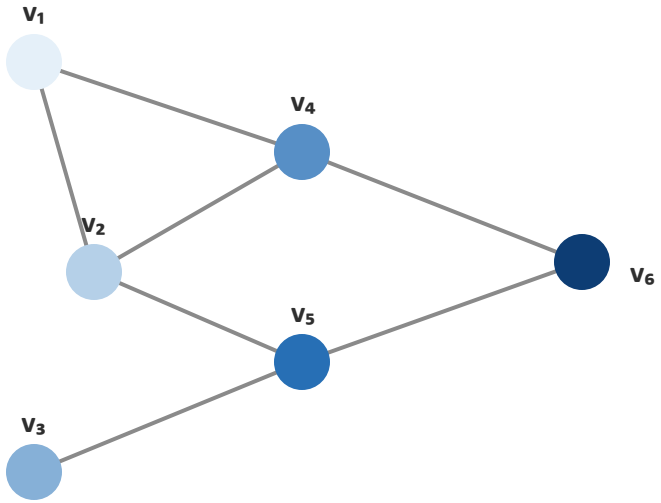
GCN and GAT work well.

Depth has two costs.

Over-smoothing

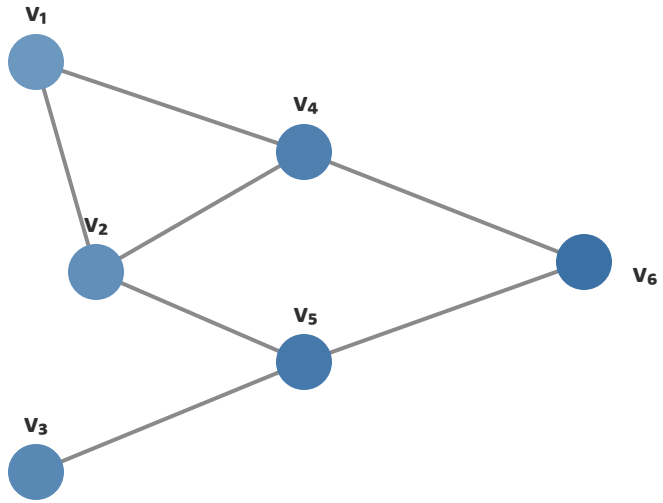
Stack too deep, and every node looks the same.

1 LAYER



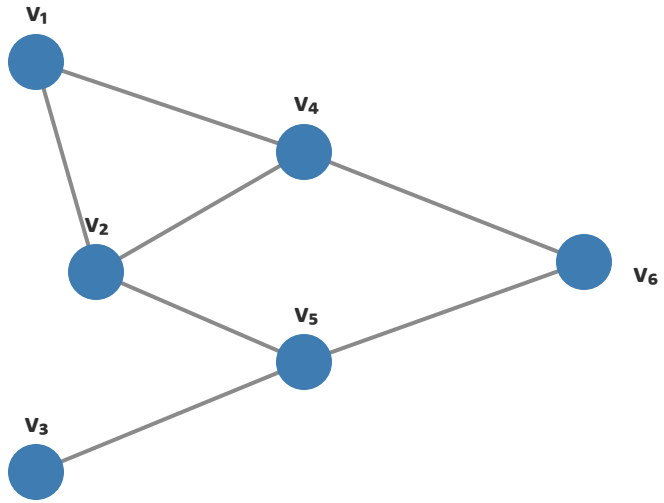
Distinct representations.

3 LAYERS



Still distinguishable, but blurring.

6 LAYERS

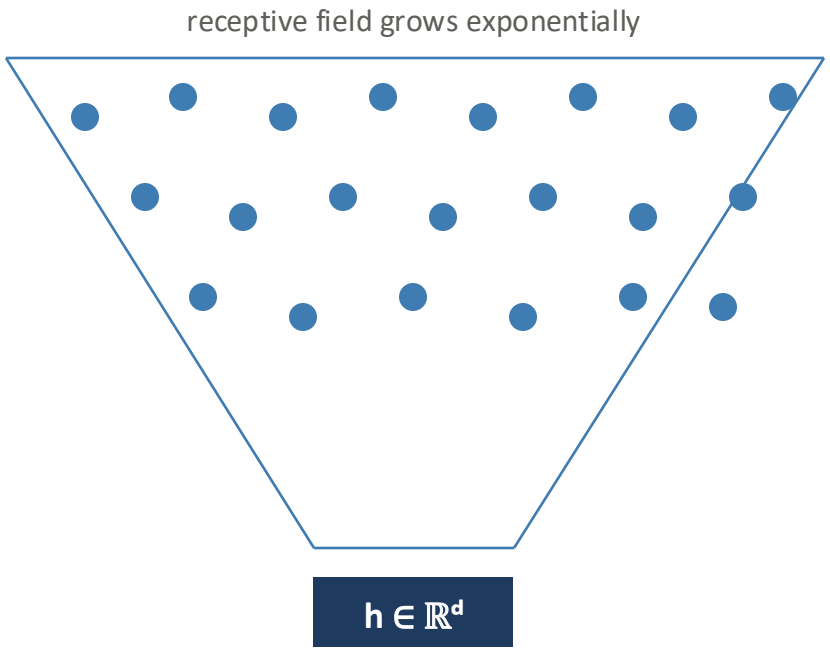


All nodes converge. Cannot classify.

Depth has a limit. Two or three layers is usually enough.

Over-squashing

The other depth problem.



WHAT IT IS

As depth grows, each node aggregates exponentially more nodes into a representation that stays a fixed size.
Long-range signals get compressed and lost.

HOW IT DIFFERS FROM OVER-SMOOTHING

Over-smoothing: representations become identical.
Over-squashing: long-range signal gets lost.

TYPICAL FIXES

Residual connections. Jumping knowledge. Graph transformers.

In practice, keep GNNs shallow.

We have the ingredients.
Let's build one.

SECTION 4

Live demonstration

Building it end to end.

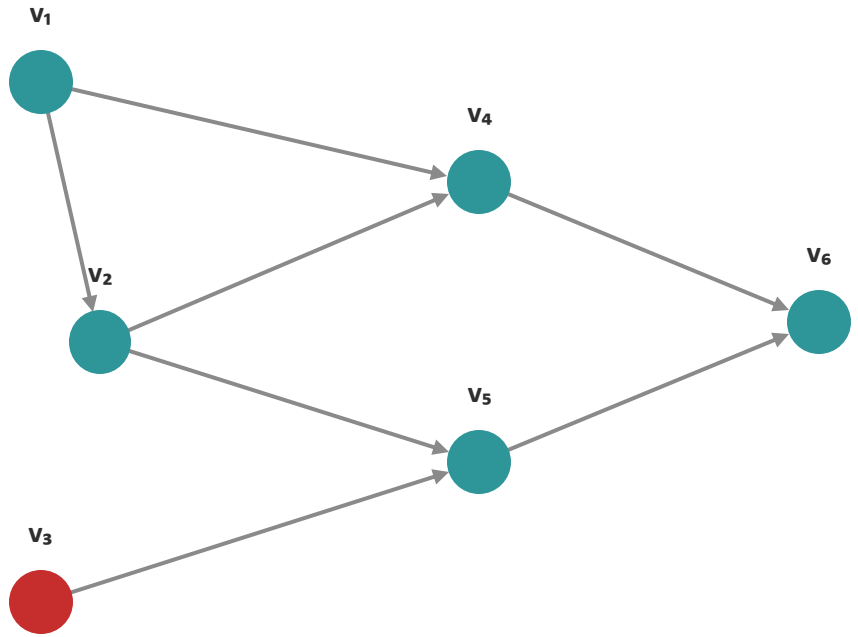
NOTEBOOK

Scan to open

Live Demo URL: https://colab.research.google.com/github/Whizz-tamie/gnn-indaba-2026/blob/main/notebooks/GNN_Tutorial_Live.ipynb

What we're building

A fraud detector on mobile money data.



legitimate fraudulent *a small piece of the real graph*

DATASET

MoMTSim

Simulated mobile money transactions.

TASK

Account-level fraud detection

Predict which accounts are fraudulent.

SCALE

13,237 accounts · 50k transactions

17.5% labelled fraudulent.

Let's train a GNN to catch the red.

From transactions to a graph

Six steps. Each is one cell in the notebook.

1	**Load**	Download the prepared sample CSV automatically.
2	**Index**	Map account IDs to 0..N integers.
3	**Edges**	Build edge_index, then convert to undirected.
4	**Features**	Aggregate per-account behaviour into X.
5	**Labels**	Flag accounts that initiated a fraudulent transaction.
6	**Assemble**	Bundle edge_index, X, y into a PyG Data object.

One cell per step. Then we train.

The baseline

An MLP on the account features. No graph at all.

	PRECISION	RECALL	ROC-AUC
MLP	0.787	0.996	0.986

One model. No graph structure at all.

This is the number the GCN has to beat.

Does it beat the baseline?

Same features. The GCN also sees the graph.

	PRECISION	RECALL	ROC-AUC
MLP	0.787	0.996	0.986
GCN	0.789	0.993	0.986

Nearly identical on every metric.

They tie. The account features already carry most of the signal.

Does structure matter alone?

Same graph. Node features replaced with random noise.

	PRECISION	RECALL	ROC-AUC
MLP	0.173	1.000	0.499
GCN	0.740	0.998	0.988

No behavioural signal in the features at all.

Educational demonstration, not production evidence — MoMTSim is synthetic, cleanly-simulated data.

MLP collapses to chance. GCN barely moves. The graph carries the signal alone.

Same frame, any graph.

SECTION 5

Beyond the demo

Where GNNs go next.

GNNs in the wild

Four African challenges. Four graphs.

1 · EPIDEMIC SURVEILLANCE

Contact networks

TB, HIV, outbreak response.
Model how disease spreads through populations.

2 · AGRICULTURE

Farm and soil networks

Yield prediction, pest and disease spread.
Farms as nodes, proximity as edges.

3 · DRUG DISCOVERY

Molecular graphs

Malaria, TB, sickle-cell.
Atoms as nodes, bonds as edges.

4 · TRANSPORT

Road and taxi networks

Urban mobility across African cities.
Traffic prediction, route optimisation.

African graphs, African problems, African solutions.

Take it further

Four ways to keep going.

1 · WATCH

Federico Barbero

Oxford GDL videos on YouTube.
Intuition for GCN and GAT.

2 · READ

Kipf & Welling (2017)
Veličković et al. (2018)

The canonical GCN and GAT papers.

3 · COURSE

Stanford CS224W
AMMI Africa GDL

Two courses. Both free online.

4 · BUILD

PyTorch Geometric

On any graph you can find.
Start with your own data.

Read one. Build one. Teach one.

Networks are everywhere.
Now you can learn from them.